SOC ANALYST PORTFOLIO CASE STUDY

Python Alert Triage and Correlation Automation

Normalizing cross-platform alert records, deduplicating mirrored events, and producing analyst-ready outputs

A 42-record mixed-source dataset was normalized into 21 unique correlated events with 21 verified mirrors and zero parser errors.

Analyst	Ryan Bryant
Exercise date	August 29, 2026
Environment	Authorized VirtualBox SOC home lab
Classification	Portfolio demonstration - no production systems

AUTHORIZED-LAB SAFETY NOTE

All scanning, authentication attempts, firewall actions, and log collection occurred inside a controlled private network owned and operated by the analyst.

Executive Summary

This lab converted Suricata and Wazuh JSONL evidence into a repeatable Python triage workflow. Four input files contained 42 raw records: 21 direct Suricata alerts and 21 corresponding Wazuh-ingested records. The script normalized field names and values, generated stable correlation identifiers, assigned triage priority, counted unique signatures, and wrote six analyst-facing outputs. Initial testing exposed a data-quality defect: direct Suricata records stored flow_id as an integer, while Wazuh rendered the same value as a string ending in .000000. Normalizing that representation reduced 42 apparent events to the correct 21 unique events, with 21 mirrored pairs and zero parser errors.

STAKEHOLDER OUTCOME

Normalization removed duplicate workload while preserving every underlying event and making the dataset immediately usable for analyst review.

Objective and Scope

- **Primary objective:** Automate routine parsing, normalization, correlation, prioritization, and reporting across two alert representations.
- **Inputs:** Four JSONL files covering 20 ICMP events and one custom HTTP event in both direct and Wazuh-ingested form.
- **Outputs:** Normalized JSONL, correlated JSON, summary JSON, signature CSV, parser-error JSON, and an analyst Markdown summary.
- **Safety boundary:** The script processed static evidence only and did not modify security controls or send alerts externally.

Processing Design

Stage	Function	Control / output
1	Discover and parse	Read four JSONL files; isolate malformed records into parse-errors.json
2	Normalize	Map Suricata and Wazuh fields into one schema and standardize data types
3	Correlate	Generate a stable identifier from event attributes and deduplicate mirrors
4	Prioritize	Assign LOW, MEDIUM, or HIGH based on controlled signature logic
5	Summarize	Write machine-readable outputs plus an analyst-facing Markdown report
6	Verify	Reconcile counts and hash the final evidence package

Correlation Defect and Root Cause

The first execution produced 42 unique identifiers and zero mirrored events even though the input intentionally contained one direct and one Wazuh representation of each event. A field-level diagnostic compared both representations and isolated the mismatch to flow_id serialization.

```
┌─[user@parrot]─[~/SOC-Labs/Python-Alert-Triage-Automation]
└─$tee notetee notes/correlation-troubleshooting.txt >/dev/null <<'EOF'
PYTHON ALERT-TRIAGE CORRELATION TROUBLESHOOTING

Initial result:
- 42 raw records
- 42 unique events
- 0 mirrored events
- 0 parser errors

Finding:
Direct Suricata records stored flow_id as an integer.
Wazuh-ingested records stored the same flow_id as a string ending in .000000.

Root cause:
The differing flow_id formats produced different correlation identifiers even though the alerts represented the same eve
nts.

Correction:
The Python normalizer was updated to remove the .000000 suffix before generating the correlation identifier.

Verified result:
- 42 raw records
- 21 unique correlated events
- 21 events mirrored between Suricata and Wazuh
- 0 parser errors

Analyst lesson:
Fields must be normalized before cross-platform event correlation. Equivalent values may use different data types or for
matting after ingestion by another security platform.
EOF

cat notes/correlation-troubleshooting.txt
PYTHON ALERT-TRIAGE CORRELATION TROUBLESHOOTING

Initial result:
- 42 raw records
- 42 unique events
- 0 mirrored events
- 0 parser errors
```

Figure 1. The troubleshooting note documents the integer-versus-.000000 flow_id mismatch and the normalization correction.

The normalizer removed a trailing .000000 suffix before correlation. This was a representation fix, not a content deletion: values that were semantically equal became equal in the correlation key.

Corrected Execution

After the correction, the run reconciled exactly: 42 raw records, 21 unique correlated events, 21 mirrored pairs, and zero parser errors. This matches the known input design of 20 ICMP events plus one custom HTTP event, each represented twice.

```
┌─[user@parrot]─[~/SOC-Labs/Python-Alert-Triage-Automation]
└──╼ $python3 -m py_compile scripts/alert_triage.py &&
python3 scripts/alert_triage.py | tee evidence/triage-execution-corrected.txt
Input files: 4
Raw records: 42
Unique correlated events: 21
Mirrored events: 21
Parse errors: 0
Outputs written to: output
┌─[user@parrot]─[~/SOC-Labs/Python-Alert-Triage-Automation]
└──╼ $
```

Figure 2. Corrected execution produced 42 raw records, 21 unique events, 21 mirrors, and zero parser errors.

```
┌─[user@parrot]─[~/SOC-Labs/Python-Alert-Triage-Automation]
└──╼ $cat output/triage-summary.json
{
  "counts_by_priority": {
    "HIGH": 0,
    "LOW": 20,
    "MEDIUM": 1
  },
  "counts_by_source_system": {
    "suricata": 21,
    "wazuh": 21
  },
  "generated_utc": "2026-08-29T18:31:33.035086+00:00",
  "input_file_count": 4,
  "input_files": [
    "suricata-custom-rule-alert.jsonl",
    "suricata-icmp-alerts.jsonl",
    "suricata-wazuh-custom-alert.jsonl",
    "suricata-wazuh-icmp-alerts.jsonl"
  ],
  "mirrored_event_count": 21,
  "parse_error_count": 0,
  "raw_record_count": 42,
  "unique_correlated_event_count": 21,
  "unique_counts_by_signature": {
    "GPL ICMP PING *NIX": 20,
    "SOC LAB Suspicious Admin Path Request": 1
  }
}
┌─[user@parrot]─[~/SOC-Labs/Python-Alert-Triage-Automation]
└──╼ $
```

Figure 3. triage-summary.json reconciles sources, priorities, signatures, and mirror counts.

Verified Results

Measure	Result	Stakeholder meaning
Input files	4	Multiple sensor representations were processed together
Raw records	42	The full input population was retained

Measure	Result	Stakeholder meaning
Unique events	21	Mirrored duplicates were removed from analyst workload
Mirrored pairs	21	Every underlying event had both expected sources
Parser errors	0	No input line was lost to malformed JSON
Priority mix	1 MEDIUM / 20 LOW	The custom HTTP detection was elevated above baseline ICMP noise

Outputs and Evidence Integrity

Output	Purpose
normalized-alerts.jsonl	Canonical record stream for downstream processing
correlated-events.json	Unique events with source/mirror context
triage-summary.json	Machine-readable counts and run summary
signature-counts.csv	Spreadsheet-ready unique signature counts
parse-errors.json	Explicit quarantine for malformed input
analyst-summary.md	Human-readable triage narrative

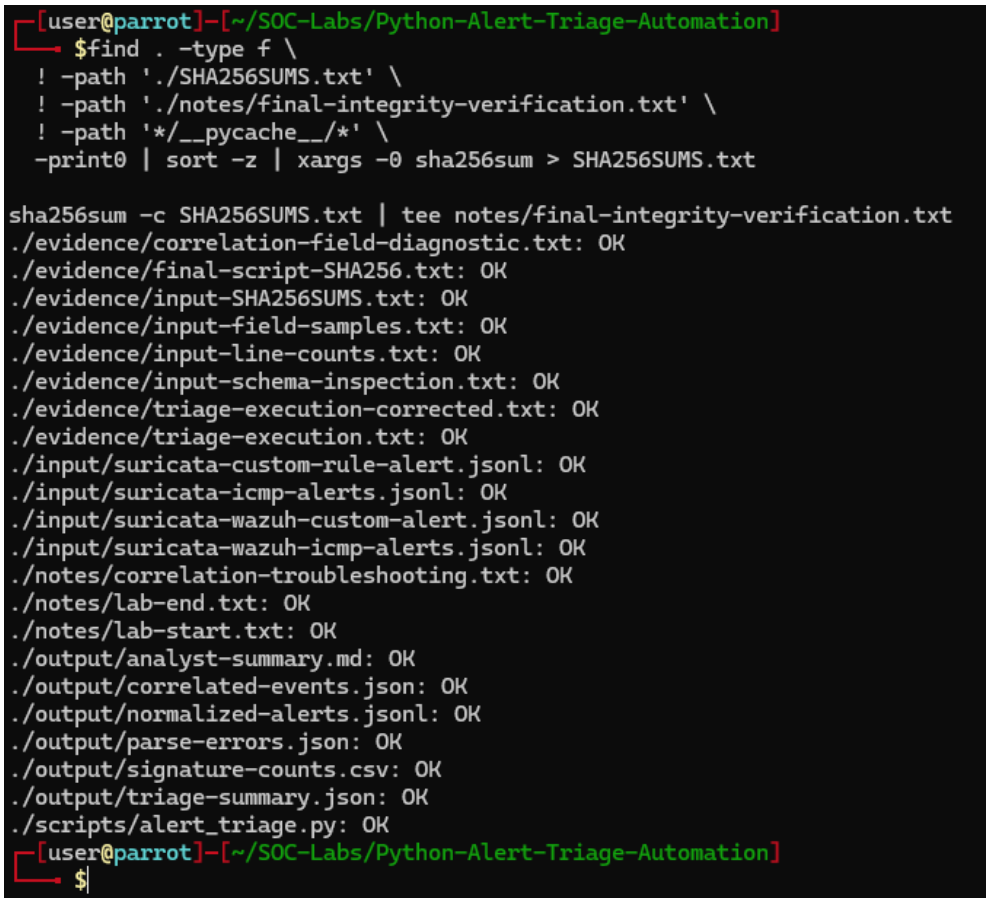

Figure 4. Full SHA-256 verification returned OK for the script, inputs, outputs, evidence, and notes.

The corrected script SHA-256 was bd80c1cd1abb21096c4b7315463a2415607c6edc5ef4d9252f561ce985e82548.

Engineering Judgment and Defensive Controls

- Kept parse failures explicit instead of silently skipping malformed lines.
- Counted unique events separately from raw records so dashboard totals would not overstate activity.
- Preserved source provenance so analysts could see whether an event appeared directly, through Wazuh, or both.
- Diagnosed the correlation defect with a focused field comparison before changing the key logic.
- Verified the final script and every material input/output with SHA-256 before archiving.

Findings and Limitations

Confirmed findings

- The automation processed all 42 records and preserved zero-error accounting.

- Normalization reduced cross-platform representation differences without collapsing distinct events in the controlled dataset.
- The output set supported both machine integration and human review.
- The priority model distinguished the custom HTTP signal from repetitive ICMP activity.

What was not proven

- The lab did not benchmark large datasets, streaming ingestion, concurrent processing, memory limits, or enterprise retention volumes.
- The priority scheme was calibrated to this controlled dataset and requires governance before production use.
- A stable identifier remains dependent on the completeness and quality of upstream fields.

Recommendations

Priority	Recommendation	Rationale
High	Add unit tests for equivalent numeric/string field representations and missing fields.	Prevents regression in the exact defect found during the lab.
High	Version the normalized schema and correlation-key definition.	Protects downstream consumers when source formats change.
Medium	Add structured logging and run metadata for scheduled execution.	Improves supportability and auditability.
Medium	Externalize triage rules into reviewed configuration.	Separates business logic from parsing code and supports controlled tuning.

Employer-Facing Project Summary

This project demonstrates defensive Python engineering: parsing JSONL safely, normalizing heterogeneous security data, building deterministic correlation, preserving provenance, reconciling counts, producing multiple analyst outputs, and troubleshooting a real data-type mismatch.

Resume Bullet

PORTFOLIO-READY BULLET

Developed a Python alert-triage pipeline that normalized 42 Suricata/Wazuh records into 21 unique events with 21 verified mirrors, zero parser errors, six analyst outputs, and complete SHA-256 integrity checks.

Interview Talking Points

- How an integer-versus-string formatting difference doubled the apparent event count.
- Why explicit parse-error and provenance outputs matter for trustworthy automation.
- How the corrected metrics were reconciled against the known lab traffic design.

Skills Demonstrated

Python | JSONL | Data normalization | Alert correlation | Deduplication | CSV reporting | Error handling | Evidence integrity

AI and Source Transparency

The analyst personally configured the virtual machines, executed the commands, validated the alerts, interpreted the results, captured the screenshots, and preserved the evidence. Generative AI assisted with command sequencing, troubleshooting suggestions, and documentation structure. Technical claims were checked against direct lab evidence and the official references listed below.

References

- Python documentation - JSON encoder and decoder. https://docs.python.org/3/library/json.html
- Python documentation - CSV file reading and writing. https://docs.python.org/3/library/csv.html
- Suricata documentation - EVE JSON format. https://docs.suricata.io/en/latest/output/eve/eve-json-format.html